FARMIQ · PRODUCT DOCUMENT

User Stories

Derived directly from the platform as built — every story below maps to real, working, tested RPCs in the FarmIQ backend, not a roadmap or wishlist.

Source: An-Agrotech-application (Auth & User service — now hosts all 7 platform domains)

Coverage: 8 gRPC services, 68 RPCs, 21 automated tests, all passing

Companion document: farmiq_mobile_api_documentation.md — RPC-level reference for implementing these stories

Date: 2026-08-20

1 Farmer — Identity & Onboarding

2 Farmer — Farms & Produce

3 Farmer — Financial Services

4 Farmer — Selling on the Marketplace

5 Farmer — Farm Intelligence

6 Farmer — Notifications

7 Buyer / Business — Account & Purchasing

8 B2B Subscriber — Data & Analytics

9 Platform — Trust & Safety

1. Farmer — Identity & Onboarding

Every other story depends on this one: a farmer needs an account, a verified identity, and a way back in before anything else is useful to them.

As a smallholder farmer, I want to create an account with my basic details and a password, so that I have a place to manage my farm and access financial services.

- Signup collects name, date of birth, home address, email, and phone — no paperwork required up front.
- The system rejects signup if my email or phone is already registered, so I don't accidentally create a duplicate account.
- My account is immediately linked to a secure login (password is hashed, never stored in plain text).

`users.UsersService/CreateFarmer`

As a returning farmer, I want to log in with my email and password and get a token that keeps me signed in, so that I don't have to re-enter my details every time I open the app.

- Wrong password is rejected without revealing whether the email exists.
- A disabled or locked account is rejected with a clear reason.

`auth.AuthDataService/FarmerLogin`

As a farmer signing up, I want to check whether my email or phone number is already taken before I fill out the whole form, so that I don't lose my progress to a duplicate-account error at the end.

`auth.AuthDataService/ValidateEmail` `ValidatePhone`

As a farmer, I want to receive a one-time code to verify my phone or email, so that the platform can trust my contact details are real and reachable.

- Codes are single-use and expire — reusing a redeemed code is rejected.

`GenerateToken` `ValidateToken`

As a farmer, I want to submit my KYC documents (ID, BVN, signature, proof of address) once, so that I can unlock loans, insurance, and marketplace selling without repeating paperwork for each service.

- I can see my overall KYC status at a glance: not started, pending, approved, or rejected.
- Status is computed honestly from my actual submitted documents — it doesn't say "approved" until every document is.

`AddFarmerKyc` `CheckKYCStatus`

Not yet built: password reset/change (the RPC exists but is a stub — there's currently no way to identify which account a password-change request belongs to). A future release needs to fix this before "forgot password" can ship.

2. Farmer — Farms & Produce

The data backbone everything else — financing, selling, telemetry, diagnosis — is built on top of.

As a farmer, I want to register each of my farms with its location, size, and what I'm growing, so that the platform understands my operation and can tailor financing, advice, and selling to it.

`AddFarm` `GetFarms`

As a farmer, I want to see my full profile — personal details, KYC, and every farm I've registered — in one place, so that I have a single view of my standing on the platform.

`GetFarmerProfile`

As a farmer, I want to record the crops/produce I have available on a specific farm — quantity, unit, price, harvest date — so that this becomes the inventory I can later list for sale.

`AddFarmProduct` `GetFarmProducts`

3. Farmer — Financial Services

The platform's largest revenue stream. These three stories are deliberately connected: your credit score changes your loan rate and your MFB referral fee — they aren't three unrelated features bolted together.

As a farmer, I want the platform to compute a credit score for me based on my verified identity, farm activity, and repayment history, so that lenders have something better than a blank slate to judge me on.

- My score improves as I complete KYC, validate my contact details, register farms and produce, and repay loans on time.
- A defaulted loan pulls my score down — the system doesn't pretend history doesn't matter.

`fintech.FintechService/ComputeCreditScore` `GetCreditScore`

As a farmer, I want to apply for an input-financing loan and get a fair, need-based interest rate, so that stronger credit history is actually rewarded with cheaper capital.

- My rate is set automatically from my credit band (12% for excellent credit up to 30% for poor/no credit) — I don't negotiate it or have it imposed arbitrarily.
- Once approved, I can see exactly what I owe in total and track it down to zero as I repay.

`ApplyForLoan` `ReviewLoanApplication` `RecordRepayment` `GetLoan` `GetLoansByFarmer`

As a farmer, I want to buy crop insurance for a farm and file a claim if something goes wrong, so that a bad season doesn't wipe me out financially.

- I can only file a claim against a policy that's currently active.
- Claims are reviewed and come back approved (with a payout amount recorded) or rejected — no claim sits in limbo indefinitely.

`CreatePolicy` `FileClaim` `ReviewClaim` `GetPolicy` `GetPoliciesByFarmer`

As a farmer with a good credit history, I want to be referred to a microfinance bank for additional financing, so that I have a path to larger capital than the platform itself offers.

- I have to have a computed credit score before a referral can happen — the platform won't vouch for me on nothing.
- The referral fee the platform earns scales with how strong my credit is, from ₦3,000 up to ₦10,000.

`ReferToMfb`

Not yet built: actual disbursement of loan funds, insurance premium collection, or claim payouts — these are all real database records today, but no payment gateway is connected anywhere in the platform yet. Treat balances and payouts as accounting figures, not confirmation that money has moved.

4. Farmer — Selling on the Marketplace

Cutting out the middleman margin the revenue model calls out as responsible for ~40% of Nigeria's harvest loss.

As a farmer, I want to list my available produce for sale with a price and quantity, so that buyers across the platform can find and purchase it directly from me.

- I can only list produce that actually belongs to one of my own farms — the platform checks this, it doesn't just trust the request.

`marketplace.MarketplaceService/CreateListing` `GetListingsBySeller`

As a farmer, I want to review and accept or reject orders placed against my listings, so that I stay in control of who I'm selling to and confirm I can actually fulfil the order.

- If I reject an order, the quantity goes back into my available stock automatically — I don't have to manually fix my inventory.

`ConfirmOrder` `GetOrdersBySeller`

As a farmer, I want to mark an order complete once I've delivered, so that the transaction is closed out and reflected in my sales history.

`CompleteOrder` `GetOrder`

5. Farmer — Farm Intelligence

Turning raw field data into decisions a farmer can act on — the free/premium hook of the freemium SaaS revenue stream.

As a farmer (or a field agent on my behalf), I want to record GPS location, sensor readings (soil moisture, temperature, humidity, rainfall), and satellite-derived vegetation health for my farm, so that this data becomes the foundation for advice, yield prediction, and eventually B2B insights.

`telemetry.TelemetryService/RecordGpsLocation` `RecordSensorReading` `RecordVegetationIndex`

As a farmer, I want a single dashboard view of my farm's latest location, vegetation health, and sensor readings, so that I can check in on my farm's status without digging through separate screens.

`GetFarmTelemetrySummary`

As a farmer, I want to describe symptoms I'm seeing on my crop (discoloration, spots, wilting, pests, stunted growth) and get a likely diagnosis with recommended action, so that I can respond to a problem before it spreads.

- If my symptoms don't clearly match a known issue, the platform tells me honestly that it can't determine a cause and to consult an agronomist — it doesn't guess just to give me an answer.

`aiml.AiMlService/DiagnoseCropIssue` `GetDiagnosisHistory`

As a farmer, I want an estimate of how much my farm will yield this season, so that I can plan sales, financing needs, and storage ahead of harvest.

- The estimate factors in my farm's size and crop type, and improves in accuracy if I have recent vegetation-health data recorded.

`PredictYield` `GetLatestYieldPrediction`

As a farmer, I want quick agronomic tips for my crop and growth stage, so that I have practical guidance even without a personal extension officer.

`GetAgronomicAdvice`

6. Farmer — Notifications

As a farmer, I want to control which channels (SMS, push, WhatsApp) the platform can reach me on, so that I only get contacted the way I actually want to be.

`notification.NotificationService/SetNotificationPreferences`
`GetNotificationPreferences`

As a farmer, I want to see a history of notifications sent to me and know how many are unread, so that I don't miss anything important about my loans, orders, or account.

`GetNotificationsByRecipient` `GetUnreadCount` `MarkAsRead`

Not yet built: nothing in the platform triggers these notifications automatically today (e.g. a loan approval does not yet send a "you're approved" alert) — the send capability exists, but the cross-domain wiring to

call it doesn't. Real SMS/push/WhatsApp delivery is also simulated, not connected to an actual gateway.

7. Buyer / Business — Account & Purchasing

Agro-dealers, food processors, and other buyers on the demand side of the marketplace.

As a business (agro-dealer, food processor, etc.), I want to register a business account with my company details, so that I can operate on the platform as a buyer.

`users.UsersService/CreateBusiness` `GetBusinessProfile`

As a buyer, I want to browse all active produce listings across the platform, so that I can find and purchase directly from farmers without a middleman.

`marketplace.MarketplaceService/GetActiveListings`

As a buyer, I want to place an order against a listing and track its status through to delivery, so that I know exactly where my purchase stands.

- I can cancel an order before it's completed, and the platform's take (its commission) is calculated transparently on every order.

`PlaceOrder` `CancelOrder` `GetOrdersByBuyer`

8. B2B Subscriber — Data & Analytics

The platform's highest-margin revenue stream: aggregated field data sold on a tiered subscription.

As an agro-dealer, I want to subscribe to the platform's data tier for my segment, so that I can access aggregated crop and market intelligence I couldn't get on my own.

- Pricing is fixed by tier, not negotiated per subscriber: agro-dealer, food-processor, and government/NGO tiers each unlock progressively more.

`analytics.AnalyticsService/CreateSubscription` `GetSubscription` `CancelSubscription`

As an agro-dealer, I want regional crop supply and pricing summaries for a state and crop type, so that I can plan sourcing and pricing decisions.

`GetRegionalCropSummary`

As a food processor, I want aggregate marketplace activity — order volume and value — so that I can gauge overall market health for the crops I depend on.

`GetMarketplaceActivitySummary`

As a government or NGO food-security stakeholder, I want farm counts and vegetation-health distribution for a region, so that I can identify areas at risk before a shortage becomes a crisis.

- This is the platform's top data tier — access is gated to subscribers at that level specifically, reflecting its higher price and sensitivity.

`GetFoodSecurityIndicators`

9. Platform — Trust & Safety

As a platform operator, I want every request beyond login/signup to require a valid, verifiable identity token, so that the API can't be called anonymously or with a stale/forged session.

- Enforced centrally for every one of the 68 RPCs across all 8 services — not something each feature has to remember to implement itself.

`JwtAuthServerInterceptor`

Honest gap: a valid token today proves *who* is calling, but not yet that they're calling *about their own* resources (e.g. that a caller updating a farm actually owns it). That per-resource authorization layer is scoped as the next major piece of trust-and-safety work.